TrendPilot — A Local Agentic AI Content Assistant

AIRI Team PITB — AI Internship Task 2

Author: Afrazia Umer Date: 13th September Submitted to: Omar Farooq, Associate AI/ML Engineer

1. What this project does

TrendPilot is a program that turns a short description of a project into a finished piece of content ready to post — a hook, a caption written for the platform you chose, hashtags, and optionally a video script and titles. It also checks its own work, rewrites it if the check fails, and saves the result to a file automatically.

You give it three things: a topic, a platform (LinkedIn, Instagram, X, YouTube Shorts), and a tone. It decides the rest by itself.

The whole thing runs on your own computer. It does not use ChatGPT or any paid service, does not need an internet connection once set up, costs nothing to run, and nothing you type ever leaves the laptop.

2. Where the "brain" came from

Instead of a photo collection, this project needed a language model — a program that already knows how to read and write. We downloaded one called `gemma3:4b` using a free tool called Ollama, which works like an app store for AI models. It is 3.3 gigabytes, small enough to run entirely inside a laptop's graphics card memory.

We tested a second, larger model as well, called `llama3.1`, before choosing `gemma3:4b`. The larger model took about a third longer per task and was not noticeably better at writing, so the smaller, faster one was kept.

The screenshot below shows Ollama itself starting up and finding the graphics card, followed by the project's own check confirming the model is installed and responding.

```
PS C:\Users\Hp\Desktop\AIRI_PITB_Internship\Task_2_Agentic_AI_TrendPilot> python demo.py --check

  Checking the environment
  ------------------------------------------------
    ok Ollama is running
    ok models installed: gemma3:4b, llama3.1:latest
    ok using gemma3:4b
    ok model replied: readyt
  ------------------------------------------------
  Environment is ready.

PS C:\Users\Hp\Desktop\AIRI_PITB_Internship\Task_2_Agentic_AI_TrendPilot>
```

3. What makes this an "agent" rather than a chatbot

A chatbot answers one question and stops. A program that always runs the same fixed steps in the same order is not really an agent either, even if it looks like one — it is just a routine wearing an agent's clothing.

This project is a genuine agent because the model itself decides what to do, not a fixed list. Ask it for just a LinkedIn post, and it skips writing a video script entirely. Ask for a video, and it includes that step. Nobody wrote down "always do these five things" — the model reads the request and writes its own short plan first, and that plan is what actually runs.

A second agent-like behaviour: after the content is written, a separate step reviews it and scores it out of five on things like how strong the hook is, whether the tone matches, and whether anything was made up that was never true. If it scores badly, the program rewrites it once using that feedback, then checks again.

4. The eight tools it can use

The task only required three. This project has eight:

- Recall past topics, so it does not repeat an angle already used
- Generate five possible angles and pick the strongest one
- Write the actual caption
- Generate hashtags
- Write a short video script, when a video was actually requested
- Suggest video titles and thumbnail text
- Review the finished content and decide if it needs fixing
- Save everything to a file

Two of these eight — recalling past topics and saving the file — do not use the AI model at all, they are just normal, fast, reliable code. That matters, because a system where every single step needs the slow AI model has not really moved past being a chatbot doing several chatbot things in a row.

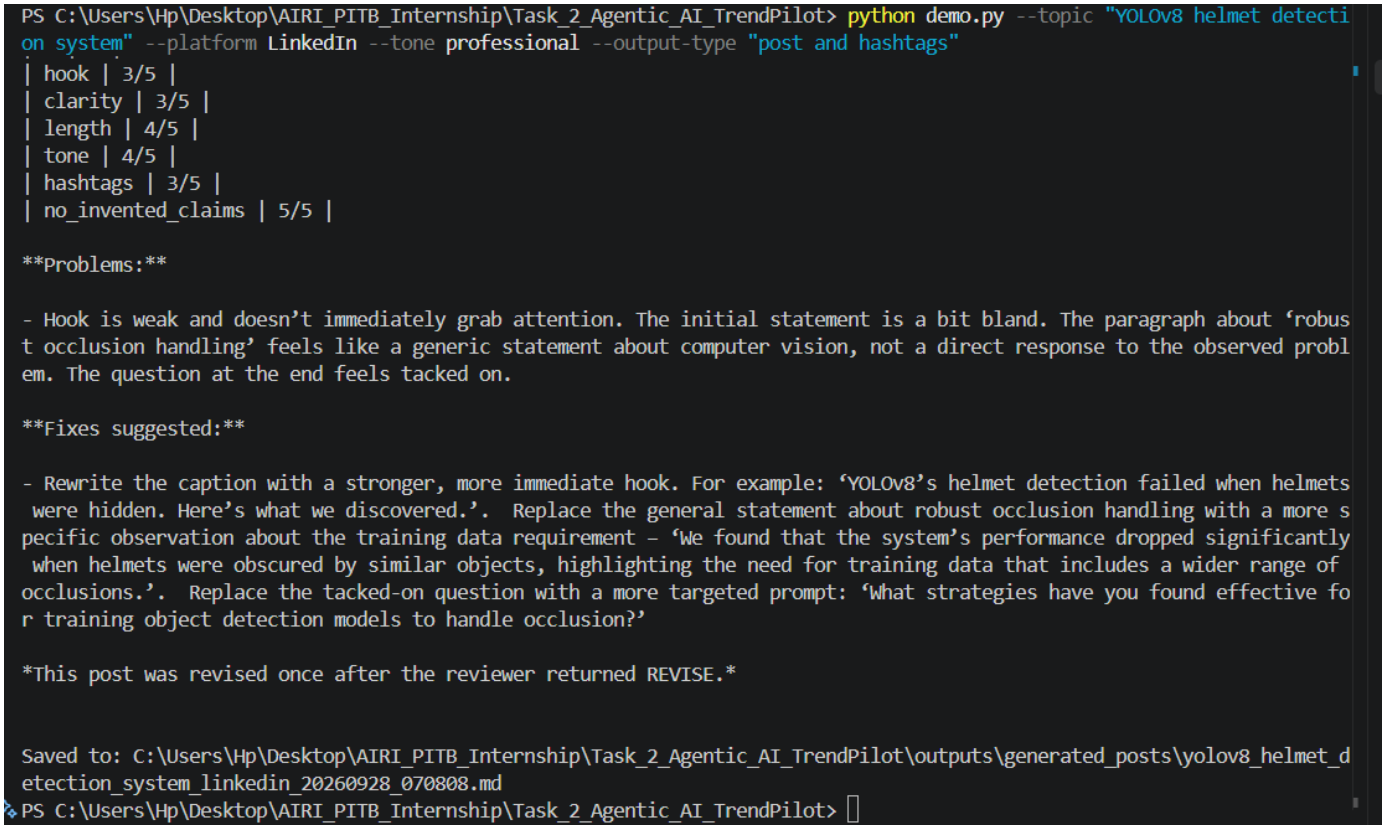
---

5. Testing it

Ten different test scenarios were run — different topics, different platforms, different tones — checking that the program produced everything it was asked for, every time, without crashing.

Result: all ten passed. Six of the ten needed one rewrite before the review step was satisfied.

The screenshot below shows one real run, asking for a LinkedIn post about a helmet-detection project. You can see the review step scoring the first draft, listing specific problems, suggesting specific fixes, and the program saving the finished file afterwards.



The next screenshot shows the folder where results get saved, across several days of testing — each file named after its topic, platform, and the exact time it was generated.

```
PS C:\Users\Hp\Desktop\AIRI_PITB_Internship\Task_2_Agentic_AI_TrendPilot> dir outputs\generated_posts
-a----        9/28/2026   3:56 AM           1424 debugging_fastapi_x_20260928_035624.md
-a----        9/28/2026   4:39 AM           1446 debugging_fastapi_x_20260928_043936.md
-a----        9/17/2026  12:18 AM           2829 final_year_project_linkedin_20260917_001828.md
-a----        9/17/2026  10:07 AM           2911 final_year_project_linkedin_20260917_100712.md
-a----        9/28/2026   3:53 AM           2992 final_year_project_linkedin_20260928_035355.md
-a----        9/28/2026   4:36 AM           2734 final_year_project_linkedin_20260928_043623.md
-a----        9/17/2026   9:30 AM           1678 machine_learning_project_x_20260917_093042.md
-a----        9/17/2026   9:51 AM           1371 machine_learning_project_x_20260917_095116.md
-a----        9/28/2026   3:49 AM            664 machine_learning_project_x_20260928_034938.md
-a----        9/28/2026   4:33 AM           1391 machine_learning_project_x_20260928_043304.md
-a----        9/17/2026  10:01 AM           1618 model_failed_in_production_instagram_20260917_100108.md
-a----        9/17/2026   9:25 AM           2520 open_source_contribution_linkedin_20260917_092528.md
-a----        9/17/2026  10:26 AM           2089 open_source_contribution_linkedin_20260917_102654.md
-a----        9/28/2026   3:57 AM           1589 open_source_contribution_linkedin_20260928_035706.md
-a----        9/28/2026   4:40 AM           3000 open_source_contribution_linkedin_20260928_044045.md
-a----        9/16/2026  11:55 PM           2176 yolov8_helmet_detection_linkedin_20260916_235557.md
-a----        9/17/2026   9:39 AM           2864 yolov8_helmet_detection_linkedin_20260917_093902.md
-a----        9/28/2026   3:45 AM           2820 yolov8_helmet_detection_linkedin_20260928_034550.md
-a----        9/28/2026   4:00 AM           2860 yolov8_helmet_detection_linkedin_20260928_040033.md
-a----        9/28/2026   4:29 AM           2498 yolov8_helmet_detection_linkedin_20260928_042930.md
-a----        9/16/2026  11:30 PM           2070 yolov8_helmet_detection_system_for_motorcycle_ri_linkedin_20260916_2
                                                 33016.md
-a----        9/16/2026  11:50 PM           1848 yolov8_helmet_detection_system_for_motorcycle_ri_linkedin_20260916_2
                                                 35011.md
-a----        9/16/2026  11:52 PM           2627 yolov8_helmet_detection_system_for_motorcycle_ri_linkedin_20260916_2
                                                 35211.md
-a----        9/28/2026   3:39 AM           2540 yolov8_helmet_detection_system_linkedin_20260928_033926.md
-a----        9/28/2026   3:40 AM           2547 yolov8_helmet_detection_system_linkedin_20260928_034053.md
-a----        9/28/2026   7:08 AM           2658 yolov8_helmet_detection_system_linkedin_20260928_070808.md
```

---

6. What went wrong, and how it was fixed

A small AI model like this one makes specific, repeatable mistakes — not random ones. Eight real problems came up while building this project, and each one taught something different.

It made up a feature that does not exist. Asked to write about a helmet-detection project, it once described it as a "real-time alert system," which it is not — it just detects helmets in photos. The instructions had said "don't invent numbers" but never said "don't invent what the project actually does." Fixed by adding a strict rule that overrides everything else: never invent a capability.

Fixing it once made it worse. One time, after a rewrite, the review score actually dropped instead of rising, because trimming the post to fix "too long" also cut out the parts that were good. Fixed by capping rewrites at one attempt and always re-checking the score afterwards, so the file that gets saved is whichever version is actually better.

It crashed with a server error mid-task. This happened because two different AI models were installed at once, and the graphics card did not have enough memory to hold both at the same time. Fixed by retrying automatically, and later by adding a step that clears out any spare model before starting.

It invented tool names that do not exist, like a made-up "viral_maximizer" tool. Fixed by checking every plan against the real list of tools and automatically correcting or dropping anything that is not real.

The auto-correction itself got confused once, matching a mistyped tool name to the wrong real tool, because several tool names share a common ending. Fixed by paying more attention to the most meaningful word in the name instead of the whole spelling.

A request for a short text post accidentally triggered a video script. The check for "did they want a video" was matching the word "short" anywhere in the request, which also appears in "short post." Fixed by matching whole words only.

Testing just two cases again accidentally deleted the results of the other eight. The test tool was overwriting the whole results file instead of updating only the parts that changed. Fixed so it merges instead.

The hardest one to track down: two video-writing tests failed repeatedly with a generic error, even after retrying automatically. Reading the actual error message hidden inside that failure showed the real cause — the model was getting

stuck repeating itself while trying to write something complicated. Testing different settings turned up something genuinely surprising: the setting literally named "repeat penalty," which sounds like it should stop repetition, did nothing at all. What actually worked was raising a different setting called temperature slightly. Fixed by detecting this specific error and automatically retrying with that adjustment.

---

7. What I learned

- Most of the actual effort was not the AI part — it was the code that catches the AI's mistakes. The plan-then-act logic itself is short; the code that repairs broken output, retries failures, and checks for invented plans is several times longer than that.
- It is easy to accidentally build something that looks like an agent but is not one. A fixed list of steps run every time would have looked identical from the outside.
- A small model's mistakes are specific and learnable, not random. Every failure found here had a clear cause and a permanent fix.
- Letting the model check its own work is not automatically safe. It reliably caught length and tone problems, but reliably missed the one failure that mattered most — making things up. One of its own corrections even made a post measurably worse.
- Running everything locally is genuinely slower than a hosted service, which is exactly why the program only runs the steps you actually asked for, rather than all eight every time.

This is a learning project completed for a one-week internship task. It runs entirely on a personal laptop using a small local AI model, and its limitations are stated here plainly rather than hidden.